

University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang



Forschungsarbeit S1470

Identifying Unknown Classes in Universal Domain Adaptation

Identifikation Unbekannter Klassen in Universal Domain Adaptation

Author: Zhengyu Bao

Date of work begin: 02.Oct.2024

Date of submission: 31.Mar.2025

Supervisor: Prof. Dr. Bin Yang

Keywords: Universal Domain Adaptation,
unknown classes, zero-shot
learning, clustering of unknown
samples, label generation

Abstract: Many universal domain adaptation methods have been applied to adapt a source model to target domain. They are able to solve both domain shift and category shift problems. However, they only count rejected samples into a single unknown class during classification. On the other hand, nowadays with foundational models it is possible to perform open-class classification or image label generation tasks with zero-shot learning. This thesis combines two topics with each other and uses zero-shot learning to push one more step after UniDA, which is unknown class identification, including unknown sample clustering and cluster label text generation.

Contents

1. Introduction	1
2. Background and Related Work	3
2.1. Universal Domain Adaptation	3
2.2. Dataset for UniDA	3
2.3. Online Source-Free UniDA	4
2.4. Zero-Shot Learning	5
3. Detection and Clustering for Unknown Samples in UniDA	7
3.1. Task Description	7
3.2. Metrics and Evaluation	8
3.3. CLIP as Feature Extractor	9
3.4. Online Clustering with DBSCAN	11
3.5. Clustering Result for Adaptation	13
3.6. Experiment Results and conclusion	14
4. Generate Labels for Unknown Samples in UniDA	17
4.1. GPT2 for Label Text Generation	17
4.2. Distance Based Label Matching	21
5. Zero-Shot Adapter for Integrated UniDA	25
5.1. Integrated Label Matching and Classification	25
5.2. More Adaptation Possibility	27
5.3. Not Distance Based Method	31
6. Conclusion	33
6.1. One-Step vs. Two-Step	33
6.2. Adaptation efficiency	35
6.3. Label Generation vs. Selection	35
A. Additionally: German Summary	37
List of Figures	39
List of Tables	41
Bibliography	43

1. Introduction

For many real-world application of deep learning methods, the distributions in feature space and label (output) space are usually not the same for samples in training set and test set, which causes domain shift between training and testing. In some cases of classification tasks, in addition to domain shift, this difference can even be shown as category shift, which means not only the distributions but also the label spaces themselves are different for training and testing. Some classes in training set may never show in test set, while in test set there may be some new classes included that have never shown during training.

For example in the application scenario of target detection and classification in autonomous driving. Different location, daytime or weather conditions will all lead to domain shift. And it is also impossible for target classification to include all the object in training, which means category shift will probably happen during application in the real word.

Many researches have been done focusing on this problem. Domain Adaptation (DA) adapts a pretrained source model to the target domain, which means the data in application is not only used for inference like a traditional test set, but also used for tuning of the model to adapt according to the distributions in target domain during application. One step further, Universal Domain Adaptation (UniDA) not only adapts the source model according to the distributions in target domain, but also allows to reject some samples and detect them as new unknown classes. [1]

Recently, one of the most important trend is that, large foundational models trained on large scale dataset have enabled zero-shot learning. [2] Different from traditional supervised learning, it is made possible with these pre-trained foundational models, to recognize or classify instances from categories which has never been seen during training. This thesis is exactly inspired by this trend and try to use Contrastive Language-Image Pretraining (CLIP), a foundational model introduced by OpenAI, to identifying unknown classes in UniDA. [3]

Particularly, during an UniDA task, data from source domain is not always accessible due to data protection or transferring regulations. [4] [5] And there are many real-world applications, requiring real-time inference of test data coming in as a stream [5]. Therefore, this thesis will focus on identifying unknown classes in this particular application case of online Source-Free Universal Domain Adaptation (SF-UniDA). Obviously, this setting is more challenging and also applicable to traditional UniDA tasks.

More background information are included in chapter 2.

To identifying unknown classes, this thesis brings up two level of identifying process. Instead of classifying all the rejected samples as one single unknown class, as what has been done in UniDA process, the very first level is to cluster all the rejected samples into several different unknown classes. Then, the second level of identifying aims to generate labels for those unknown clusters as a basic description for those samples which is rejected as not belonging to known classes in source domain.

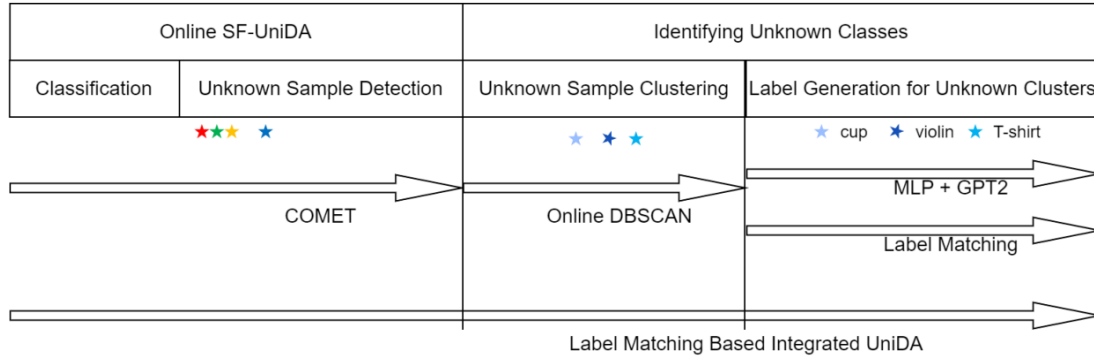


Figure 1.1.: Identifying Unknown Classes: A Workflow

Figure 1.1 shows two different processing approach for these two levels of unknown class identifying.

The first concept is separating the clustering and label generation process and take a two-step solution.

After applying a contrastive mean teacher (COMET), which is a powerful tool for online SF-UniDA, [5] all the samples rejected as unknown are first clustered into different unknown clusters. This process will be described in detail in chapter 3.

Then two different methods are applied to generate labels for unknown clusters, which is GPT2-based open-dictionary label generation and distance-based label candidates selection. They are included in chapter 4.

And the second concept fully uses the zero-shot learning with CLIP, to integrate two levels of unknown class identifying, clustering and label generation, as one-step solution. It is introduced in chapter 5.

Finally, in chapter 6 an conclusion will be given.

2. Background and Related Work

2.1. Universal Domain Adaptation

As described in the introduction chapter, in the practical application of deep learning methods, there are usually domain shift and category shift between training in source domain and testing in target domain, which ask for Domain Adaptation (DA) methods.

In the frame of Closed-set DA, there is no category shift. Classes are the same for source domain and target domain $C_t = C_s$. While Open-set DA is defined for the situation, that some classes in target domain have never shown in source domain. In other words, the target label space is a superset of the source label space $C_t \supset C_s$. And a Partial DA defines the opposite situation that target label space is a subset of the source label space $C_t \subset C_s$. [4]

Universal Domain Adaptation is first brought up to combine Closed-set DA, Open-set DA and Partial DA. The source label set and target label set share a common label set and both hold a private label set at the same time. [1]

During the adaptation in the target domain, UniDA requires a model not only to classify a target sample correctly if it belongs to those classes that have shown in source domain, but also to mark those target sample as “unknown”, if they belong to those classes that only show in target domain but not in source domain. [1]

2.2. Dataset for UniDA

DomainNet is the biggest and most popular dataset regarding to domain adaptation tasks. It contains six domains and about 0.6 million images distributed among 345 categories. [6] The following figure 2.1 shows directly the domain shift for the same class across different domains.

In our thesis, DomainNet is used to set up a UniDA task. Besides of domain shift which is already included in the dataset, we select particular classes for source domain and target domain to ensure that they have common classes and private classes at the same time.

In detail, the classes are chosen as table 2.1.

Table 2.1.: select classes from DomainNet to set up category shift [5]

	Source Domain	Target Domain
Class Index	0-199	0-149, 200-344

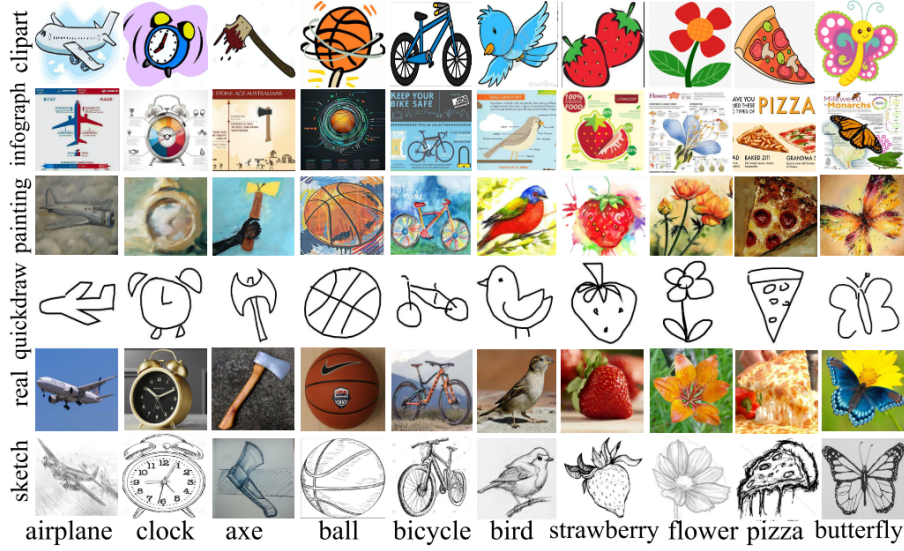


Figure 2.1.: 6 domain of DomainNet, reproduced from [6]

From the table we can also be informed that the “unknown” samples in the target domain have ground truth label index from 200 to 344. In traditional UniDA tasks, their ground truth are set as an index for the same unknown class (eg. 345). [5] However in this thesis, their ground truth are also used to evaluate the clustering performance and accuracy of generated label.

The 2017 Visual Domain Adaptation (VisDA-2017) dataset is another popular dataset for domain adaptation. It was the biggest dataset for domain adaptation up to then, including 280K images across 12 categories in the training, validation and testing domains. [7] Similarly, the classes are selected as table 2.2

Table 2.2.: select classes from VisDA to set up category shift [5]

	Source Domain	Target Domain
Class Index	0-8	0-5, 9-11

The description and argument in chapter 3 - 5 are based on painting domain in DomainNet as source domain and real domain as target. Results of the experiments based on the other datas can be found in chapter 6.

2.3. Online Source-Free UniDA

As mentioned in the introduction, this thesis handles mainly with online source-free UniDA. Source free is defined that source and target samples can not be accessed in the same stage. After the pre-training based only on source domain, during the adaptation, no access to the previously seen source samples is allowed. [4] It doesn't mean that no information from source domain can be used. For example in chapter 5, the label text from source domain is used. But in this case, it is still a source-free approach.

And online scenario means that, data comes as a stream and is processed with individual batches. [5] Adaptation is performed only based on the current batch and requires real-time result. And each batch is only allowed to be processed once. In other words, once one step adaptation is done, the adaptation result can not be influenced by later batches.

The requirements of source-free and online scenario are not only for UniDA tasks, but also for unknown sample clustering and label generation after traditional UniDA.

The method COMET uses a contrastive mean teacher for the online source-free UniDA which is used in chapter 3 of this thesis to finish the classification and unknown sample detection tasks.

2.4. Zero-Shot Learning

Since unknown classes have never shown during training on source domain, and we still want to perform unknown samples clustering, a model which has zero-shot learning ability is needed.

Here zero-shot learning doesn't refer to no training on the source domain, but only means that the features of different unknown classes is obtained without learning from source domain.

Contrastive Language-Image Pre-training (CLIP) from OpenAI is such a possibility. It is trained based on 400 million image-text pairs and have strong zero-shot transfer performance. Its ability to encode image and text into the same 512 dimensions feature space also give us the potential to perform text related task like label generation. [3] This will be further described in the following chapters.

3. Detection and Clustering for Unknown Samples in UniDA

In order to deal with category shift between source domain and target domain, UniDA methods are able to reject some samples in target domain during classification and count them as unknown. However, after the unknown sample detection, traditional UniDA methods don't include any further processing for those samples which are detected as unknown.

This chapter will focus on the detection and clustering of unknown samples. This task will be described in more detail in section 3.1 and 3.2. Section 3.3 and 3.4 will introduce how to apply the clustering method DBSCAN in a online scenario, where test data come as a stream and ask for real-time processing.

Once the clustering for unknown samples is done, the features of different unknown clusters can also be learned by the model as a part of adaptation, which is one step even further after clustering and could help with the unknown sample detection in UniDA. This will included in section 3.5.

3.1. Task Description

During UniDA problems, the target label space $\mathcal{Y}_t$ are different as the source label space $\mathcal{Y}_s$. Therefore, some samples in the target domain are rejected during classification and are counted as unknown class. For example in this chapter, a contrastive mean teacher (COMET) is used for online SF-UniDA [5], in which the unknown class is reprecented with class index $|\mathcal{Y}_s| + 1$.

After traditional UniDA process, here COMET as an example, the samples in a batch from target domain $\{x_i^t, y_i\}_{i=1}^{N_b}$ are classified as $\{\hat{y}_i\}_{i=1}^{N_b}$, with:

$$y_i \in \mathcal{Y}_t, \quad \hat{y}_i \in \mathcal{Y}_s \cup \{|\mathcal{Y}_s| + 1\} \\ \text{and} \quad \mathcal{Y}_t \neq \mathcal{Y}_s$$

And those samples $\{(x_i^t, y_i) \mid \hat{y}_i = |\mathcal{Y}_s| + 1\}$ are defined as **Detected Unknown Samples**.

Because the number of true unknown classes in target domain is not pre-obtained, the task is to **form unknown clusters** for all the unknown samples:

$$C_{\text{unknown}} = \text{Clustering}(\{(x_i^t, y_i) \mid \hat{y}_i = |\mathcal{Y}_s| + 1\})$$

However, in a online scenario, where test data come as a stream and are processed with batches, it makes the task of unknown sample clustering is more difficult.

First, one of the biggest challenges is that the number of unknown samples in one single batch is very limited. And online clustering requires real-time solution based on this limited number of unknown samples.

Meanwhile, two unknown samples with the same true label $y_i = y_j (y_i, y_j \in \mathcal{Y}_t)$ could show in different test batches, while test batches can only be accessed one at a time and each batch is only available once. This brings the other challenge which is trying to assign those samples with the same cluster number.

3.2. Metrics and Evaluation

To evaluate the performance of the unknown sample clustering, several definitions are given as following: **step noise samples**

During the unknown sample clustering, some unknown samples are not clustered with others, but are assigned with an individual cluster number. These samples are defined as noisy samples.

Also to mention is that, because the number of detected unknown samples is very limited for one batch. If only those samples alone are clustered and evaluated, there would be many noise samples. Therefore, for an unknown sample in one batch, whose cluster number is although individual in the current batch, will not be counted as noise sample, if its cluster number is the same as some samples in previous batches. This definition also encourages to form cluster across different batches, even each batch is only available for processing once.

sample pair

Different from classification, the evaluation of clustering performance is based on sample pairs.

In our case, if a batch has $N_{b, \text{unknown}}$ unknown samples, the number of sample pairs is:

$$\binom{N_{b, \text{unknown}}}{2} = N_{b, \text{unknown}} * (N_{b, \text{unknown}} - 1) / 2$$

precision and recall

During a clustering, true positive is defined as that, a pair of samples are assigned with the same cluster number and they belong exactly to the same ground-truth class. And false positive is defined as that, a pair of samples are wrongly assigned with the same cluster number, while they belong to different ground-truth classes.

In this case, precision shows, the proportion of sample pairs that belong to the same ground-truth class among all sample pairs assigned to the same cluster.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Similar as the definition of precision, recall is defined as, the proportion of sample pairs that are assigned to the same cluster, among all sample pairs that belong to the same ground-truth class.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

And here false negative is defined as that, a pair of samples belong to the same ground-truth classes, but they are wrongly assigned with different cluster number.

latest precision and recall

In our online scenario, data comes as a stream and is processed with batches. However, as mentioned, the unknown cluster forming should cross different batches. In this case, after each data batch, precision and recall are calculated for all processed samples, including samples from both current batch and previous batches. Therefore the latest precision and recall are calculated to evaluate the performance of unknown cluster forming across different batches up to particular step.

In practical, to remember all the assigned cluster number during adaptation history and use them to calculate latest precision and recall, will cost both memory and huge computation effort. Therefore, in this thesis, only the result of latest 200 samples are evaluate. In other words, precision and recall between latest 200 samples are calculated as latest precision and latest recall.

This definition helps to save computation complexity, and is enough to evaluate the performance of unknown cluster forming across different batches during an adaptation process. It also helps with focusing on the recent performance.

total precision, recall and f1-score

The total precision and recall are defined as the average value for latest precision and recall over all batches in the target domain, from which a f1-score can be calculated

latest and total noise ratio

Noise ratio is the proportion of noisy samples in all the detected unknown samples, which is also an important metrics to evaluate clustering performance. latest noise ration is defined as the noise ratio within one batch. And total noise ratio is calculated based on all the detected unknown samples after all batches in an adaptation process.

metrics for unknown samples detection

The performance of unknown samples clustering depends also on the performance of unknown sample detection. Therefore, the metrics for unknown samples detection are also taken into consideration when unknown sample clustering is evaluated.

They are known accuracy, unknown accuracy and H-score.

3.3. CLIP as Feature Extractor

As described in section 3.1, in this chapter, before the unknown sample clustering, the method COMET is used for online SF-UniDA. A source model which consists of a ResNet backbone, an one-layer feature extractor and a classifier is trained on source domain and is used for adaptation on target domain. [5] The source model is able to rejected unknown samples and to learn the features of the unknown samples through contrastive loss.

However, with the help of contrastive loss, which compares the features of unknown samples with known classes, the model is only able to learn the difference between unknown samples and known classes. This method does not have the ability to learn the features between

different unknown classes in target domain, which makes it very difficult for unknown sample clustering.

In other words, those unknown samples, which belongs to new unknown classes in target domain, have never shown in source domain during pre-training. Because of this, in order to still implement clustering for unknown samples, zero-shot learning is needed. Zero-shot learning means, even though the new unknown classes in target domain have never shown in source domain during pre-training due to category shift, the model would still have the ability to learn the feature of unknown samples and to tell the difference between them.

Therefore, in order to obtain zero-shot learning ability, the ResNet backbone is replaced with CLIP. In this way, as the figure 3.1 shows, the source model has a CLIP backbone, an one-layer feature extractor and a classifier.

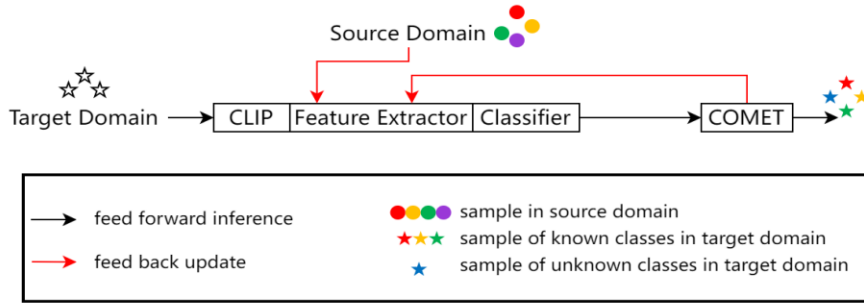


Figure 3.1.: CLIP used as backbone to draw features from images

Generally, for an image sample $(\mathbf{x}_0, y)$, CLIP is used to draw features vectors $\mathbf{x}_1$ from these image samples:

$$\mathbf{x}_1 = \text{CLIP}(\mathbf{x}_0) \quad \text{with : } \mathbf{x}_1 \in \mathbb{R}^{512}$$

After CLIP, an one-layer feature extractor is used to reduce the feature dimension from 512 to 216:

$$\mathbf{x}_2 = \text{ReLU}(W_1 \mathbf{x}_1 + b) \quad \text{with : } W_1 \in \mathbb{R}^{256 \times 512}, \quad b \in \mathbb{R}^{256}, \quad \mathbf{x}_2 \in \mathbb{R}^{256}$$

Finally, a linear classifier is used:

$$\mathbf{x}_3 = W_2 \mathbf{x}_2 + b \quad \text{with : } W_2 \in \mathbb{R}^{C \times 256}, \quad b \in \mathbb{R}^C, \quad C = |\mathcal{Y}_s| + 1$$

As shown on figure 3.1, the model is firstly pre-trained on the source domain. During the pre-training of the source model, only the one-layer feature extractor and classifier are trained, while the parameters in CLIP are frozen from updating.

And during the adaptation on target domain, with the help of COMET, parameters in the one-layer feature extractor are updated for adaptation. [5] The CLIP backbone still remains frozen.

Even CLIP used as backbone is frozen, it is still able to provide zero-shot learning ability and help with the further processing of unknown sample clustering. This is proved by the result in section 3.6.

3.4. Online Clustering with DBSCAN

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a popular method for clustering which is based on the distance between samples in the feature space. A distance as similarity threshold ϵ and minimal sample points number in a cluster $n_{\min}$ are both used to control the process of clusters forming. [8]

In the method of DBSCAN, all the sample points are divided as core points, border points and noise points. A core point is a point, from which at least $n_{\min}$ samples (including itself) can be found within the distance of ϵ in the feature space. A border point itself is not a core point, but is located with a distance to core point less than ϵ . And those which are not core points nor border points, are noise points. [8]

The figure 3.2 shows a simple example of using DBSCAN for clustering, with ϵ and $n_{\min} = 4$. In the figure, there are two clusters formed for all the samples. The red cluster has 4 core points, while the blue cluster has 2. The decision boundary of two clusters are drawn with red or blue circles with a radius of ϵ . It is shown, that with the method DBSCAN, not only spherical clusters but also any arbitrarily-shaped clusters can be formed. Besides, there are also 3 noise samples.

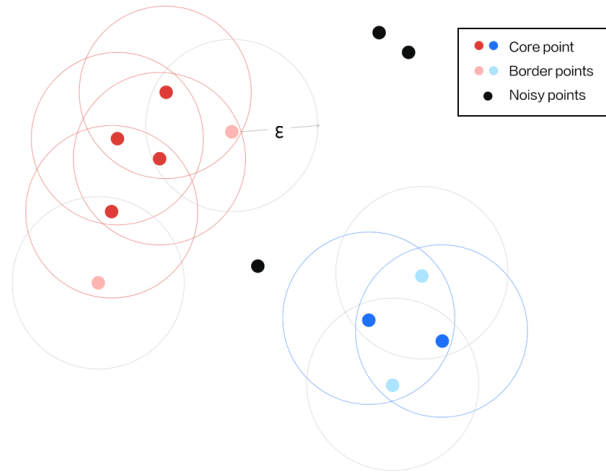


Figure 3.2.: An example of DBSCAN in 2-dimensional feature space

And as section 3.1 described, we still need to **implement DBSCAN in online scenario**, where test data come as a stream and are processed with batches. A concept is shown on the figure 3.3.

In order to form clusters across different batches and reduces the number of noise samples, the information from previous batches should be remembered for the the current being processed batch. But to remember all the previous sample points is unrealistic. Therefore, for those already formed cluster, instead of remembering all the sample points for further clustering, a cluster prototype which is a mean value of all its sample feature vectors is used.

In the mean time, to still prevent from memory going to infinity, a leaky bucket is set for remembering the noisy sample. And it uses a first in first out (FIFO) strategy, which means each noise sample is only temporarily remembered. If it is not included in any clusters even if it has been remembered for a while, then it is forgotten.

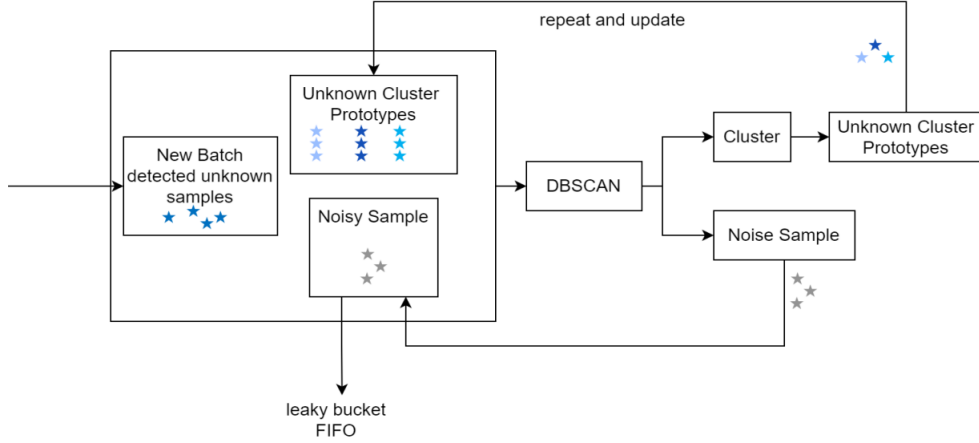


Figure 3.3.: Online Clustering with DBSCAN

In detail, the online clustering with DBSCAN starts after unknown sample detection.

As notated in section 3.1 the detected unknown samples of the batch b in target domain $\mathcal{X}^{t, b} = \{(x_i^{t, b}, y_i) \mid \hat{y}_i = |\mathcal{Y}_s| + 1\}$ are to be clustered.

Here $x_i^{t, b} \in \mathbb{R}^{256}$ is the feature vector of a unknown sample after CLIP and one-layer feature extractor.

The DBSCAN clustering is implemented for the combination set of detected unknown samples in the current batch, together with remembered unknown cluster prototypes $\mathcal{X}^{c, b-1} = \{x_i^{c, b-1}\}_{i=1}^{|\mathcal{C}_{\text{unknown}, b-1}|}$ and noise samples $\mathcal{X}^{\text{noise}, b-1} = \{x_i^{\text{noise}, b-1}\}_{i=1}^{N_{\text{noise unknown}, b-1}}$ from batch $b-1$.

$$\mathcal{C}_{\text{unknown}, b}, \{x_i^{\text{noise}, b}\} = \text{DBSCAN}(\mathcal{X}^{t, b} \cup \mathcal{X}^{c, b-1} \cup \mathcal{X}^{\text{noise}, b-1})$$

After the DBSCAN clustering for the batch b , the number of noise unknown samples $N_{\text{noise unknown}, b} = |\{x_i^{\text{noise}, b}\}|$ is compared with the size of leaky bucket N_{bucket} . If the size of leaky bucket is smaller, then only last N_{bucket} of the noise samples are remembered:

$$\{x_i^{\text{noise}, b}\} = \{x_i^{\text{noise}, b}\}_{i=N_{\text{noise unknown}, b}-N_{\text{bucket}}+1}^{N_{\text{noise unknown}, b}}$$

As for those formed unknown cluster, the cluster prototype $x_i^{c, b}$ for each cluster with index c are calculated and remembered.

$$x_i^{c, b} = \text{mean}(\{x_i^b \mid \hat{y}_i' = c\}) \quad \text{with : } \hat{y}_i', c \in \mathcal{C}_{\text{unknown}, b}$$

In this formula, mean value of all the features in the combined feature set during DBSCAN, whose cluster estimates $\hat{y}_i'$ are equal to cluster index c_i , is calculated as the prototype for this unknown cluster.

Finally, the unknown cluster prototypes $\{x_i^{c, b}\}$ are repeated for $n_{\min}$ times, so that after the combining of detected unknown samples in the next batch $b+1$, together with current remembered unknown cluster prototypes and noise samples, the prototypes are sure to form a cluster during DBSCAN clustering.

With this structure, cluster is able to include samples from different batches with only a limited memory. The experiment result and some more discussion can be found in section 3.6.

3.5. Clustering Result for Adaptation

Once the clustering for unknown samples has been done, this information could also be very useful for processing of unknown samples in further data stream. During the source training and UniDA process with COMET, the one-layer feature extractor and classifier are trained to learn the features of known classes and difference between known classes and unknown samples. But up to now, there are not any adaptation method to learn the features and tell the difference between different unknown clusters. It is also shown on the figure 3.4, where all the model parameters updating processes are marked with red lines.

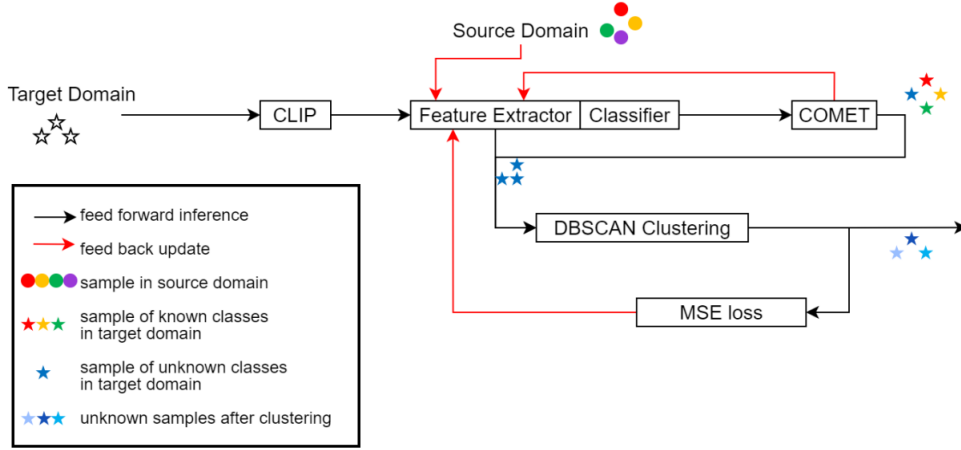


Figure 3.4.: Using unknown sample clustering result for adaptation

As this figure also shows, after the clustering of unknown samples, the model first has the chance to learn the features and tell the difference between different unknown clusters. This helps not only with the unknown sample clustering for further batches, because the clustering bases on the feature vectors after the one-layer feature extractor. It may also help with the unknown sample detection.

In detail, MSE loss is used to minimize the distance between the feature of clustered unknown sample and the unknown cluster prototypes as following:

$$\mathcal{L}_{\text{mse}} = \frac{1}{|\mathcal{U}_c|} \sum_{i \in \mathcal{U}_c, j \in \mathcal{C}_{\text{unknown}, b}} \|x_i^b - x_j^{c, b}\|_2^2$$

with $\mathcal{U}_c = \{x_i^b \mid \hat{y}_i' \in \mathcal{C}_{\text{unknown}, b}\}$

In this formula, the squared euclidean distance of all the clustered unknown sample to their unknown cluster prototypes are calculated and used as MSE loss.

As for noise samples, on the one hand, our goal is to reduce noise ratio. On the other hand, most of the noise samples are not really 'noise' but only not be clustered due to online scenario. Therefore, instead of implementing contrasitive learning like in the COMET method, they are just ignored during this step of adaptation.

The experiment results of using clustering information for adaptation can be found in section 3.6.

3.6. Experiment Results and conclusion

The table 3.1 shows the result of hyperparameter tuning for online clustering with DBSCAN which is introduced in section 3.4.

There are 2 hyperparameters during clustering with DBSCAN, the distance ϵ and the minimum sample number in a cluster $n_{\min}$. The result shows, with larger distance and smaller $n_{\min}$ the noise ratio reduces but the accuracy of clustering also reduces. And these 2 hyperparameter only influence the unknown sample clustering. The classification and unknown sample detection stage are not influenced.

Table 3.1.: hyperparameters tuning for DBSCAN based unknown sample clustering

Hyperparameters			UniDA acc			Unknown Clusters			
backbone	ϵ	$n_{\min}$	known	unknown	H-score	precision	recall	f1-score	noise ratio
CLIP	5	4	56.1%	67.3%	61.2%	79.9%	81.2%	80.1%	73.8%
CLIP	5	3	56.1%	67.3%	61.2%	71.4%	72.3%	71.9%	72.1%
CLIP	4	4	56.1%	67.3%	61.2%	72.3%	66.1%	69.0%	88.1%
CLIP	6	4	56.1%	67.3%	61.2%	50.8%	70.6%	59.1%	66.3%
Resnet	12	4	48.5%	62.0%	54.4%	18.0%	81.4%	29.4%	90.5%
Resnet	15	4	48.5%	62.0%	54.4%	0.7%	89.3%	1.4%	60.0%

As introduced in section 3.3, the original Resnet backbone is changed with CLIP to encode an image to obtain zero-shot learning ability. Here zero-shot learning means that even when unknown classes have never shown in source domain, the source model is still able to tell the difference between them.

The results in the table 3.1 also prove this. It shows when Resnet is used and contrastive learning is performed during COMET method, the unknown samples are clustered as one single unknown class. Because during DBSCAN, if we use a small distance $\epsilon = 12$, almost all the detected unknown samples are noise. While we raise the distance $\epsilon = 15$ to reduce the noise ratio, then the clustering accuracy immediately reduces to 0.

This means a source model with Resnet backbone does not have zero shot ability and is not able to tell the difference between different unknown classes.

The result in table 3.1 base on a online clustering with DBSCAN without leaky bucket. As described in section 3.4, the leaky bucket is used to forget long term noise sample. That means, a noise sample in one batch is only temporary memorised for clustering in further batches. Without a leaky bucket, the memory and computation complexity could go to infinity due to the accumulation of noise samples.

For example, after the entire adaptation process with CLIP backbone and $\epsilon = 5, n_{\min} = 4$, from painting domain in DomainNet as source to real domain as target, the number of accumulated noise sample is over 31 k, which requires huge computation complexity during DBSCAN.

The following table shows the result when different size of leaky buckets are used. They all uses CLIP as backbone and $\epsilon = 5, n_{\min} = 4$.

Table 3.2.: Performance of unknown sample clustering after UniDA

bucket size	UniDA acc			Unknown Clusters			
	known	unknown	H-score	precision	recall	f1-score	noise ratio
∞ (no bucket)	56.1%	67.3%	61.2%	79.9%	81.2%	80.1%	73.8%
5000	56.1%	67.3%	61.2%	86.4%	93.5%	89.8%	76.5%
500	56.1%	67.3%	61.2%	83.7%	96.5%	89.6%	89.6%

As expected, when a leaky bucket with a proper size is used, the noise ratio will only slightly raises. Only when the bucket size is too small, it will influence the performance.

Surprisingly, a leaky bucket also helps with better clustering accuracy. It may be because that the long term noise sample themselves are more difficult to be properly clustered. If they are forgotten, the performance will be better although the noise ratio is slightly higher.

Section 3.5 also brought up that, the unknown sample clustering result can be used by feature extractor to learn features from different unknown classes. In this way, during each batch in the data stream, the parameters in one-layer feature extractor are updated twice. Once for contrastive learning between known and unknown classes and the second time for learning from unknown clustering result. Its result is shown on the following table 3.3. For the experiment result in this table, a leaky bucket with size of 5000 is used.

Table 3.3.: unknown sample clustering result for learning

Hyperparameters			UniDA acc			Unknown Clusters			
2. learning	ϵ	$n_{\min}$	known	unknown	H-score	precision	recall	f1-score	noise ratio
no	5	4	56.1%	67.3%	61.2%	86.4%	93.5%	89.8%	76.5%
yes	5	4	55.7%	67.3%	61.0%	86.4%	94.3%	90.2%	76.6%
no	6	4	55.9%	67.0%	60.9%	89.3%	63.3%	74.1%	71.6%
yes	6	4	55.7%	67.5%	61.0%	89.2%	61.8%	73.0%	71.6%

The result shows that the improvement is quite little. This may be because only single epoch adaptation is not enough for the one-layer feature extractor with 131 k trainable parameter to learn the features between different unknown classes.

In conclusion, by implementing DBSCAN in an online scenario, cluster is able to include detected unknown samples from different batches. Together with unknown cluster prototype and leaky bucket for noise sample, the clustering is performed with a limited memory and computation complexity.

However, there is still an issues which is high noise ratio. The following figure 3.5 shows the noise ratio for batches during an adaptation.

At the beginning batches of an adaptation, because the number of detected unknown samples is very limited, the noise ratio is very high. Since we remember all the noise sample and use them to form cluster for later batches, the noise ratio goes down gradually. But it still remains stable afterwards at around 75% and the leaky bucket is also full. This issue can not be solved even if a bigger ϵ is used.

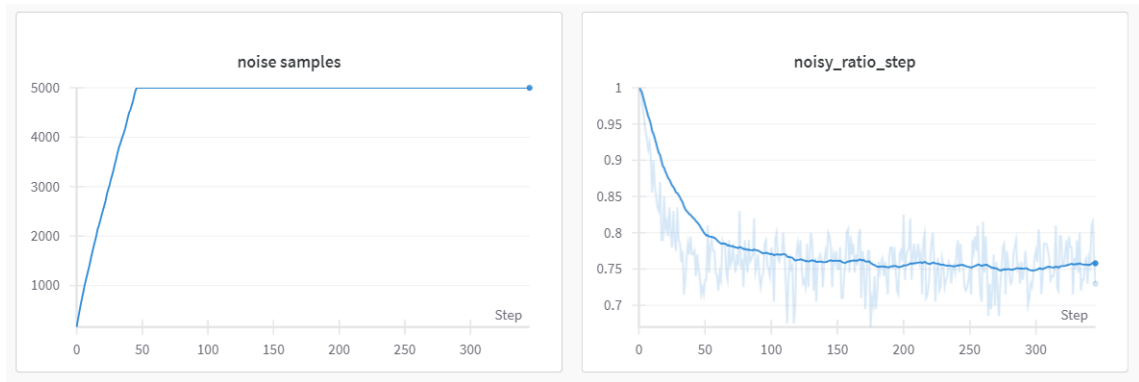


Figure 3.5.: noise ratio remains high during unknown sample clustering

This may be because of the disadvantage of implementing DBSCAN in online scenario. One big disadvantage of this solution is that, when only the unknown cluster prototypes are updated and remembered, the method DBSCAN will lose its ability to form arbitrarily-shaped clusters. Further improvement can be achieved by GMM based adaptation.

4. Generate Labels for Unknown Samples in UniDA

After chapter 3, the unknown samples are clustered and the single unknown class now is identified as several unknown clusters. In order to obtain more information about these clusters, this chapter will discuss the possibility to generate labels for them as figure 4.1 shows.

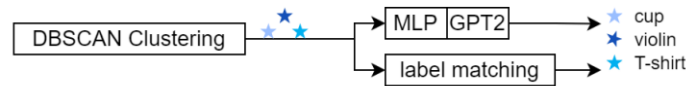


Figure 4.1.: Generate labels for unknown clusters after DBSCAN

In this chapter, 2 different methods are proposed. section 4.1 will introduce how to use a large language model (LLM) to generate labels. While the other approach in 4.2. is to first organize a dictionary as label candidates, from which a proper label can be selected for unknown clusters.

Also to mention is that, in chapter 3, feature vectors $\mathbf{x} \in \mathbb{R}^{256}$ after the one-layer feature extractor are used in DBSCAN for clustering. However, in order to first evaluate the ability of label generation more directly and to enable a more direct training process for the model. Here in chapter 4, the feature vectors $\mathbf{x} \in \mathbb{R}^{512}$ from CLIP output are used instead. The further discussion on this topic can be found in the section 6.1.

Label generation with LLM and label matching from candidates dictionary, in the first glance, the concept of these two method seem very different. One needs a candidates set while the other is more like a open generation. However, during tokenization of a LLM, the vocabulary is also needed. And a candidates set can also grow to a larger size. It is argued that, the boundary between generation and selection is actually not so clear as it seems. This topic will be discussed with more detail in section 5.3 and 6.3.

4.1. GPT2 for Label Text Generation

In recent years, with the development of large language model (LLM), many researches has been done to generate text from images. For example, ClipCap is a method where a mapping network is trained to map the encodings between CLIP and GPT2, so that GPT2 can generate captions for images. [9] [10] This section makes an adjustment to this concept and force the GPT2 to generate labels instead of free-style captions for images.

First, the definitions of **token and embedding in GPT2** is here clarified to understand the input of the GPT2:

Token and embedding are used to encode texts into a feature space. During this text encoding process, the input texts are first divided into a sequence of words or subwords. These minimum 'units' of texts are defined as tokens. For example, in the vocabulary of GPT2, there are 50257 tokens in total. And embedding process transfers each token to a high dimensional vector, which finally transfers texts (sequence of tokens) into a sequence of vectors. As for GPT2, the embedding space has 768 dimensions. [10]

Then we can **organize our feature mapping network**. After being processed by CLIP, an image is encoded as a 512-dimensional vector which contains an overview information of this image. Exactly the same information should be used as input of GPT2 for label generation.

It is assumed in the method ClipCap, that the scale of 10 tokens is enough to contain the information in an image which could be used for captioning. [9] Because in our case only a label but not a caption is asked, we could use smaller scaled tokens to represent an image. And this number of tokens is actually a hyperparameter which can be tuned later during training. But firstly we just keep this size of 10 tokens as unchanged.

A Multi-Layer Perceptron (MLP) is used to take the output 512-dimensional vector from CLIP, implement mapping process and transfer it to a 768-dimensional vector sequence that has a length of 10. With this mapping network this sequence is reorganized as token embeddings and is as an input for GPT2 for label generation.

During the training, CLIP and GPT2 are frozen from parameter updating. Only the MLP need to be trained. However, even in the simplest case, that the mapping network can be just two dense layers, if there are 1024 neurons in the hidden layer, there are still:

$$512 * 1024 + 1024 + 1024 * 768 * 10 + 768 * 10 = 8,397,312$$

8M trainable parameters in total, which means the mapping network needs huge scale of training.

In order to **force GPT2 to generate labels** instead of free style captions, the output of GPT2 should also be reorganized. The inference of GPT2 is an iterative process. In each iteration, it predicts and generates the next token. And the generated token is again appended to the input sequence for next generation iteration.

The method ClipCap limits GPT2 to maximumly generate 50 tokens. [9] But in some case, GPT2 just predicts an end token which stops the generation. While in our case, a label usually need 2 tokens. Due to this reason, GPT2 is set to finish after only 2 generation iteration, so that it is forced to generate a label.

Besides of the limitation to the output, a prompt is also created to encourage GPT2 to generate only labels. The input of the GPT2 is not only the 10 token embeddings from MLP mapping network. Instead, those 10 of 768-dimensional embedding vectors are added behind a prompt which is 'Please generate a label for the image: '. Together with the 10 token embeddings which include the information of the image sample, the prompt as an input for GPT2 also help us to get label as generation.

Finally, **a cross entropy loss is defined for the training**.

For an image sample $(\mathbf{x}_0, \mathbf{y})$ in training:

$$\begin{aligned}\mathbf{x}_1 &= CLIP(\mathbf{x}_0) \quad \text{with : } \mathbf{x}_1 \in \mathbb{R}^{512} \\ \mathbf{x}_2 &= MLP(\mathbf{x}_1) \quad \text{with : } \mathbf{x}_1 \in \mathbb{R}^{768 \times 10}\end{aligned}$$

After 2 generation iteration of GPT2:

$$\begin{aligned}\mathbf{z}_1 &= GPT2(\text{tokenize}(\text{'Please generate a label for the image:'}); \mathbf{x}_2) \quad \text{with : } \mathbf{z}_1 \in \mathbb{R}^{50257} \\ \mathbf{z}_2 &= GPT2(\text{tokenize}(\text{'Please generate a label for the image:'}); \mathbf{x}_2; \mathbf{z}_1) \quad \text{with : } \mathbf{z}_2 \in \mathbb{R}^{50257}\end{aligned}$$

As mentioned before, 50257 is the vocabulary size of GPT2. And $\mathbf{z}_1, \mathbf{z}_2$ is the 2 logits generated by GPT2, from which a softmax can be performed:

$$\mathbf{P}_i = \text{softmax}(\mathbf{z}_i), \quad i = 1, 2$$

And the cross entropy loss is calculated as:

$$\mathcal{L}_{\text{sample}} = -\frac{1}{2} \sum_{i=1}^2 \log p_i(y_i), \quad \text{with : } (y_1, y_2) = \mathbf{y}$$

This formula also shows that two ground truth token y_1, y_2 is needed to calculate their cross entropy with $\mathbf{P}_1$ and $\mathbf{P}_2$. And due to the character of LLM which is not deterministic result, the generation process is repeated 3 times for each sample.

The training is on ImageNet [11], which may be the only choice when such large scale of data is needed. However, the true labels still need to be processed to get the ground truth token. In a short phrase, the central noun is often placed after the attributive modifiers. Therefore, if the tokenization of a true label from ImageNet longer than 2, only the last 2 tokens are taken as ground truth tokens.

Figure 4.2 shows the result of a training procedure. Due to the large scale of training, the procedure shows the first 4 epoch. For each epoch over 640,000 samples from 1000 classes are available. And the training for one epoch lasted more than 100 hours.

From these first 4 epoches of the training, we can tell that both the training and validation loss continues to reduce, while their accuracy continues to raise. However, the training efficiency remains very low. And the averaged accuracy is lower than 2%.

The low accuracy is also because of the evaluation method. In our approach, a correct generation is defined as both 2 generated tokens match with the ground truth token. Some cases happened during the training like some images of 'book store' are labeled as 'library' and 'goldfish' are labeled as 'fish', which is very convincing but counted as wrong.

The cross entropy loss also slows the training procedure. A CE loss bases on each token among 50257 tokens in GPT2. Learning progress for one particular ground truth token is very hard to share to global distributions. In this way, it is more like a LLM is guessing true value among a large candidate, and is only able to learn from the close guesses.

To solve this issue, the author argued and tried to use CLIP again, to encode and draw features from ground truth label and generated label, from which a cosine similarity loss can be



Figure 4.2.: Training procedure of a MLP between CLIP and GPT2

calculated. In this way, the training in the beginning should be more efficient and accuracy calculation can include more tolerance.

The combination of two different loss should be perfect. Because the cosine similarity loss speed up the training procedure while the cross entropy loss encourages to generate exactly the same label with the ground truth. However, there was very little difference in performance as the experiment result shows on figure 4.3.

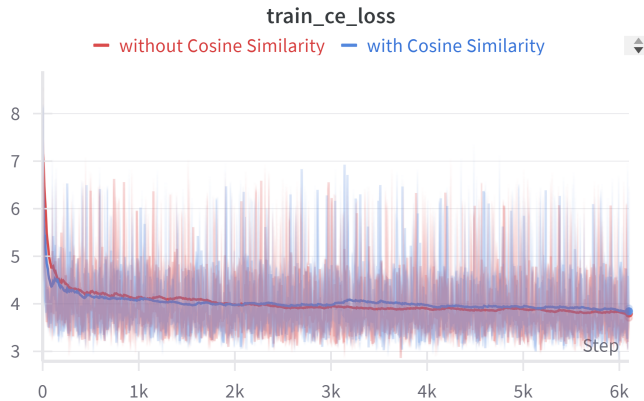


Figure 4.3.: Try to speed up training procedure by cosine similarity loss

In conclusion, this method has certain ability for candidates-free label generation, but needs large scale of training, which is expensive and low efficient.

As for the training itself, it involves other tricks like continuells learning, self-supervised learning and so on. An improvement of training method for such a CLIP involved model can be found in section 5.3.

4.2. Distance Based Label Matching

Up to now in this thesis, the CLIP model is only used to analyse images. In chapter 3, the images are processed by CLIP to get features for clustering, while in section 4.1. this features from CLIP are used by GPT2 for label generation. However, when it comes to lebel generation which is a language tack, it can not be ignored that CLIP itself has a strong ability to process and encode texts.

Therefore, in this section 4.2. it will be discussed, how to only use CLIP model to develop a more practical and low-cost method for label generation.

For example, the research Classifier-Oriented Calibration via Textual Prototype (COCA) tried to form clusters for an open set of images and asign labels with CLIP. After k-mean clustering, COCA uses CLIP to select a label from a candidates dictionary for the fromed clusters based on distance in feature space. Then, this result are used for self-supervised learning of a classifer. [12]

Inspired by COCA, this section will evaluate the performance of distance based label matching by CLIP. However, similar as section 4.1. for more direct evaluate, the label matching process will be implemented for individule image samples but not clusters. In this way, only the ability of label matching process are evaluated.

CLIP is a model which has only encoding ability, which means it can draw high dimensional features from images and texts and project them in the same feature space. However, CLIP itself can not generate. The label matching process is defined as selecting proper label from a candidates set. As mentioned in the very beginning of chapter 4, the difference and connections between label generation and label matching will be included in the section 6.3.

In this way, the **label matching process** is defined as followings:

For an image sample $(\mathbf{x}_0, \mathbf{y})$, the CLIP is used to extract feature from it:

$$\mathbf{x}_1 = CLIP(\mathbf{x}_0) \quad with : \mathbf{x}_1 \in \mathbb{R}^{512}$$

Then the cosine similarity in CLIP feature space from the image sample to each label from candidates set $\mathcal{Z} = \{\mathbf{z}_c^{\text{text}}\}_{c=1}^C$ with the candidates number of $C = |\mathcal{Z}|$, is calculated:

$$\cos(\mathbf{x}_1, \mathbf{z}_c^{\text{text}}) = \frac{\mathbf{x}_1 \cdot \mathbf{z}_c^{\text{text}}}{\|\mathbf{x}_1\| \cdot \|\mathbf{z}_c^{\text{text}}\|}$$

For the label with index c , $\mathbf{z}_c^{\text{text}} \in \mathbb{R}^{512}$ is its feature vector in CLIP feature space. The text encoding uses only the label text but also a prompt. For example for the label ‘dog’, the text ‘This is a picture of dog’ is encoded by CLIP.

And the label with the highest cosine similarity is chosen:

$$\hat{c} = \arg \max_c \cos(\mathbf{x}_1, \mathbf{z}_c^{\text{text}})$$

In practical the top 3 labels are selected to calculate the top-3 accuracy.

Besides, **how to pick a dictionary** is also a big challenge. The labels from source domain are obtained. However, in order to generate labels for unknown samples, a larger dictionary is needed.

The dictionary is built up based on WordNet, which is a lexical database for English. It is not a simple dictionary, but a lexical network based on cognitive linguistics. The words in WordNet are organized according to semantic relations to form a hierarchical lexical structure, which can be used to filter out the abstract nouns and select only concrete noun for our dictionary. [13]

In detail, a score system is developed to select words. If a noun is related to ‘physical entity’, ‘artifact’, ‘plant’ and so on, it will get higher score for selection. While it is related to ‘abstraction’, ‘attribute’, ‘psychological feature’ and so on, it will get lower score.

Besides of the score system, those words whose scores are higher than threshold is sorted by their frequency of usage. This frequency is counted in two popular linguistic corpuses Brown and Gutenberg.

Finally the most frequently used concrete nouns are selected as a candidates set.

The following 2 tabels 5.1 and 4.2 shows the result of distance based label matching. As mentioned, in this experiment, we don’t only use known samples, but try to select label for every individule image in the dataset. In this way, only the process of label matching is evaluated.

Table 4.1.: Labels cover rate of dataset by candidates

n-candidates	5000	10000	20000	40000
DomainNet	57.39%	69.28%	79.71%	86.09%

Table 4.2.: Top-3 accuracy of label matching

n-candidates	5000	10000	20000	40000
DomainNet-Real	40.25%	42.94%	37.86%	27.19%
DomainNet-Painting	29.25%	28.94%	25.78%	19.68%
Visda-train	37.21%	28.29%	11.33%	7.31%

The result shows that it is still very difficult to cover most ground truth label texts by a candidates dictionary. A dictionary with 10k candidate labels is only able to achieve a cover rate of 69.28%. On the other hand when the dictionary size goes over 20k, the label matching performance will be influenced. To achieve a higher cover rate, a combination of dictionary and label space from source domain would also help. This is introduced in chapter 5.

Coming from this result, it is argued that a classifier is redundant. For example in the method COCA, label matching result is used as pseodo label to train a classifer. However, if we use all the class text from DomainNet as a very small candidate set and perform a distance based label matching, the top 3 accuracy is over 90%. The tabel 4.2 also shows, even if a candidates dictionary with a size of 10000 candidate labels is used, which has a ground truth

cover rate of 69.28%, the label matching accuracy for DomainNet real domain is 42.94%. This means the label matching accuracy for those samples, whose ground truths are covered by candidates, is 62.0%, which is already higher than the H-score achieved by a classifier with CLIP backbone in chapter 3.

Therefore, instead of implementing label matching only for detected unknown samples, it becomes more direct and efficient, to integrate classification task into label matching. This is described in the next chapter 5.

5. Zero-Shot Adapter for Integrated UniDA

As described in chapter 4, the good performance of CLIP in label matching for images makes it possible to combine the classification and label generation. In this case a classifier is not needed anymore, the clustering for unknown samples either. The process of classification, unknown sample detection as well as unknown clusters forming are all integrated during label matching process, which is defined as integrated UniDA in this chapter

This integrated UniDA will be introduced in section 5.1. And section 5.2 will show, how this method can be made more adaptable.

5.1. Integrated Label Matching and Classification

If a label matching process has a good enough performance, whose accuracy is even higher than a classifier. Then the classification can be performed based on the result of label matching. And a classifier seems in this way redundant. It is exactly the case in this section, we try to integrate the classification, as well as unknown sample detection and clustering based on the label matching result.

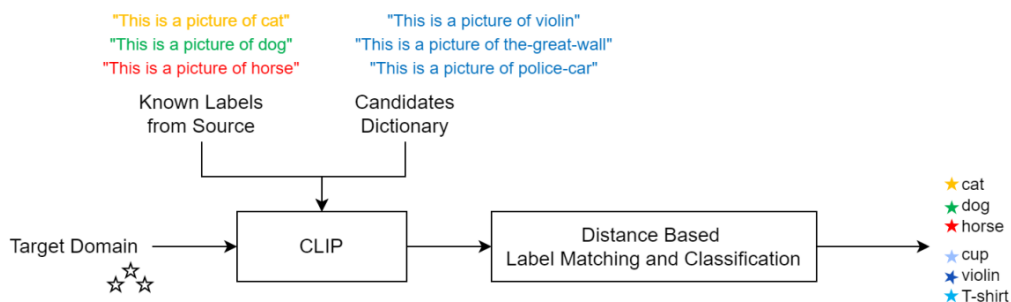


Figure 5.1.: label matching in UniDA problems

As introduced in section 4.2., distance based label matching with CLIP need a candidates set, from which the proper label can be selected. And we used different sets which have from 5000 up to 40000 candidate labels, because in section 4.2 we assume that no knowledge of unknown samles are pre-known.

While especially for integrated UniDA problems in this chapter, the label matching is not only implemented for unknown samples identification, but also for the classification and unknown sample detection. In this case, compared with a large candidates dictionary, the labels from source domain can be a smaller candidates set which is more helpful for classification and

rejecting samples as unknown. Therefore, as shown on the figure 5.1, two different candidates dictionary are used. One includes all the labels from source domain, while the other bigger one works as candidate labels for the rejected unknown samples.

The classification, unknown samples detection and clustering are integrated with distance based label matching as following in detail:

For a sample in the target domain $(\mathbf{x}_0^t, y)$, the first step is still to encode it into CLIP feature space:

$$\mathbf{x}_1^t = \text{CLIP}(\mathbf{x}_0^t) \quad \text{with : } \mathbf{x}_1^t \in \mathbb{R}^{512}$$

Then, the first try of label matching is done based on the small dictionary $\mathcal{Z}_{\text{known}} = \{\mathbf{z}_c^{\text{text}}\}_{c \in C_{\text{known}}}$ which includes all the label index from source domain C_{known} . And $\mathbf{z}_c^{\text{text}}$ is the encode of a label text with index c together with a prompt “this is a picture of” by CLIP. Similar as section 4.2 this process is based on cosine similarity.

$$\cos \theta_{\max,1} = \max \cos(\mathbf{x}_1^t, \mathbf{z}_c^{\text{text}}) \quad \text{and} \quad \hat{c}_1 = \arg \max_c \cos(\mathbf{x}_1^t, \mathbf{z}_c^{\text{text}})$$

$$\text{with : } \mathbf{z}_c^{\text{text}} \in \mathcal{Z}_{\text{known}}$$

After the first label matching, a threshold $\theta_{\text{confidence, known}}$ for cosine similarity is drawn to get known samples with higher confidence.

For those samples which have not been decided yet, if they belong to known classes or should be rejected as unknown, a second label matching process is taken with the larger candidates dictionary $\mathcal{Z}_{\text{unknown}} = \{\mathbf{z}_{c'}^{\text{text}}\}_{c' \in C_{\text{unknown}}}$ which includes candidate labels for unknown samples C_{unknown} .

$$\cos \theta_{\max,2} = \max \cos(\mathbf{x}_1^t, \mathbf{z}_{c'}^{\text{text}}) \quad \text{and} \quad \hat{c}_2 = \arg \max_{c'} \cos(\mathbf{x}_1^t, \mathbf{z}_{c'}^{\text{text}})$$

$$\text{with : } \mathbf{z}_{c'}^{\text{text}} \in \mathcal{Z}_{\text{unknown}}$$

And $\cos \theta_{\max,1}$ and $\cos \theta_{\max,2}$ are compared to confirm if the sample belongs to unknown classes or should be rejected as unknown. A ratio α is used to control the decision boundary.

The decision process are shown as:

$$\begin{aligned} &\text{if } (\cos \theta_{\max,1} \geq \theta_{\text{confidence, known}}) \quad \text{then: } \hat{c} = \hat{c}_1 \in C_{\text{known}} \\ &\text{if } (\cos \theta_{\max,1} < \theta_{\text{confidence, known}}) \text{ and } (\cos \theta_{\max,1} \geq \alpha * \cos \theta_{\max,2}) \quad \text{then: } \hat{c} = \hat{c}_1 \in C_{\text{known}} \\ &\text{if } (\cos \theta_{\max,1} < \theta_{\text{confidence, known}}) \text{ and } (\cos \theta_{\max,1} < \alpha * \cos \theta_{\max,2}) \quad \text{then: } \hat{c} = \hat{c}_2 \in C_{\text{unknown}} \end{aligned}$$

In this way, computation complexity is also saved from the second label matching for those confident known sample.

The result of hyperparameter tuning for integrated UniDA is shown on the tabel 5.1 :

As the result shows when the ratio α is set as smaller as 1, the samples will be more probably classified into known classes, which helps with more accurate unknown sample detection. It also helps to get a more stable result because CLIP as a foundational model has a non-deterministic output.

Table 5.1.: Performance of integrated UniDA

Hyperparameters			UniDA acc			Unknown Clusters			
θ	α	$ C_{\text{unknown}} $	known	unknown	H-score	precision	recall	f1-score	label acc
0.3	0.95	10000	67.3%	84.9%	75.1%	77.4%	29.1%	42.3%	19.3%
0.3	1	10000	46.6%	94.1%	62.3%	72.4%	23.5%	35.5%	19.2%
0.3	0.95	20000	58.9%	90.0%	71.2%	83.4%	18.4%	30.1%	14.5%
0.3	0.95	5000	72.4%	75.1%	73.7%	76.1%	36.7%	49.5%	25.3%
0.25	0.95	10000	79.0%	36.0%	49.5%	81.3%	25.1%	38.3%	16.9%
0.35	0.95	10000	66.1%	86.7%	75.0%	76.9%	29.2%	42.3%	19.4%

As for the confidence threshold $\theta_{\text{confidence, known}}$, when it is equal to 1, then it means label matching processes are performed twice for each sample, once with labels from source domain and the other time with candidates dictionary. And no computation complexity is saved. On the other hand, when it is equal to 0, all the samples are confidently classified into known classes. The result table shows, 0.3 is a proper value to select confident known samples and to save computation at the same time.

The size of the candidates dictionary makes also a difference. A smaller dictionary helps to achieve better performance during unknown sample clustering. Because as mentioned, it helps with the problem of a too scattered clustering and raises in this way the total recall. However, when the size of the candidates dictionary is not big enough, the performance of classification and unknown sample detection may be influenced. In our case, a candidates dictionary with 10000 candidate labels achieves highest H-score.

Finally, after hyperparameters tuning, the integrated UniDA achieves better performance than UniDA method COMET during the classification and unknown sample detection stage. For unknown sample clustering, it also solve the problem of high noise ratio in DBSCAN.

However, the table also shows that, when the unknown sample clustering is performed based on label matching, precision is usually much higher than recall. This means, due to the big size of the candidates dictionary, the samples are assigned with too many different labels and form too many clusters. To solve this problem, more adaptation should be done, so that those labels, which have been matched for any previous samples, can have higher possibility to be selected for later samples. In this way, the samples are more clustered and recall can therefore be higher. These adaptation ability will be introduced in the next section.

The other important point to mention is that, integrated UniDA fully uses zero-shot learning ability from CLIP. The only needed information from source domain is the label texts. And none of samples in source domain is needed, not for pretraining nor for adaptation. In the other word, it can adapt to any dataset, even if we only have partial knowledge about the true categories in this dataset.

5.2. More Adaptation Possibility

The method of integrated UniDA, which integrates classification, unknown sample detection and unknown sample clustering with label matching process, is able to perform adaptation to

the category shift in target domain. This means with the help of a large candidates dictionary, a sample in target domain can be rejected as unknown and classified into particular unknown classes.

However, it uses CLIP feature space and can not adapt to the domain shift which is the particular samples distribution of a class in target domain. Because the CLIP model is too big to perform adaptation based on small scale of data, especially in the online scenario where data comes and is processed as batches in a stream.

Therefore, in this section, a linear adaptation layer is brought up for more adaptation possibility. This concept is shown on the figure 5.2.

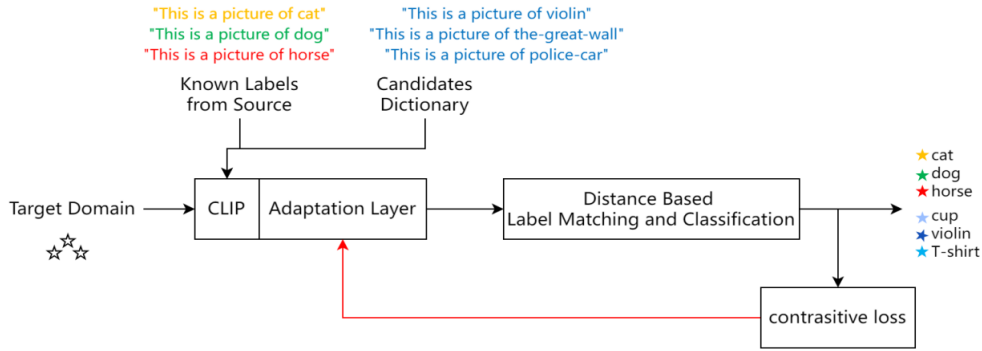


Figure 5.2.: More adaptation possibility by contrastive learning

The distance based label matching and classification still use the CLIP feature space. However, the feature of the samples in target domain is aligned after direct CLIP encoder. To achieve this, a linear adaptation layer is introduced. Meanwhile, with a special initialization method, this adaptation layer will not destroy the zero-shot learning ability of CLIP model before the adaptation.

For a sample in the target domain $(\mathbf{x}_0^t, y)$ and its feature in CLIP feature space $\mathbf{x}_1^t = \text{CLIP}(\mathbf{x}_0^t) \in \mathbb{R}^{512}$, the adaptation layer performs linear feature alignment:

$$\mathbf{x}_2^t = \mathbf{W}\mathbf{x}_1^t + \mathbf{b}$$

with : $\mathbf{x}_1^t, \mathbf{x}_2^t, \mathbf{b} \in \mathbb{R}^{512}$ and $\mathbf{W} \in \mathbb{R}^{512 \times 512}$

And the weights and biases in this layer are initialized with a method which is similar as one-hot coding:

$$\mathbf{W} = \mathbf{I} \in \mathbb{R}^{512 \times 512}$$

$$\mathbf{b} = \mathbf{0} \in \mathbb{R}^{512}$$

So that before the adaptation, the input and the output of the linear adaptation layer are exactly the same. And the zero-shot learning ability of CLIP model is remained:

$$\begin{aligned} \mathbf{x}_2^t &= \mathbf{W}\mathbf{x}_1^t + \mathbf{b} \\ &= \mathbf{I}\mathbf{x}_1^t + \mathbf{0} \\ &= \mathbf{x}_1^t \end{aligned}$$

Now even when the CLIP is frozen, we still have some parameters to update. And a loss is still to be defined. Since we already calculated the cosine similarity between the sample and known labels during label matching process. The cosine similarity loss is in this case the most direct way.

Inspired by the method of COMET, the first concept is to push known and unknown samples away from each other with contrastive loss. In this way we minimize the cosine similarity loss for known samples and maximize the cosine similarity loss for unknown samples. Here the cosine similarity is calculated between samples and known labels in the first label matching.

And for training, only those samples with higher confidence are used. Therefore, together with the confidence threshold $\theta_{\text{confidence, known}}$, another threshold $\theta_{\text{confidence, unknown}}$ is introduced to select confident unknown samples.

Meanwhile, in order to prevent from pushing unknown samples to infinity and squessing unknown classes with each other, a “safe distance” $\epsilon_{\text{unknown margin}}$ is defined. If the unknown sample located already beyond safe distance, then its loss should be set as zero.

The loss for a sample $(\mathbf{x}_0^t, y)$, the loss is calculated is as following in detail:

if: $(\cos \theta_{\text{max},1} \geq \theta_{\text{confidence, known}})$
 then: $\mathcal{L} = 1 - \cos \theta_{\text{max},1}$
 else if: $(\cos \theta_{\text{max},1} < \theta_{\text{confidence, unknown}})$ and $(\cos \theta_{\text{max},1} > \epsilon_{\text{unknown margin}})$
 then: $\mathcal{L} = \cos \theta_{\text{max},1}$
 else: $\mathcal{L} = 0$

With contrastive loss, the feature vectors of known samples are drew closer to the CLIP encode of known labels while the feature vectors of unknown samples are pushed farther. However, the result in the table 5.2 shows that this contrastive learning doesn’t help much with unknown sample detection. The improvement of H-score is very limited.

It is argued that two classes which can not separate linearly, is still not linearly seperatable after only linear feature mapping. During our contrastive learning for adaptation, only the soft margin is getting larger. For this reason, the improvement of H-score is very little.

But the result shows that the f1-score and accuracy of label matching are improved with contrastive loss based adaptation. However, this improvement is based on a higher unknown accuracy during unknown sample detection, which is caused by decision boundary shifting. It is very hard to evaluate the improvement from contrastive learning.

Table 5.2.: Performance of integrated UniDA with adaptation

adaptation method	UniDA acc			Unknown Clusters			
	known	unknown	H-score	precision	recall	f1-score	label acc
without	67.3%	84.9%	75.1%	77.4%	29.1%	42.3%	19.3%
contrastive loss	67.1%	85.7%	75.3%	77.7%	30.0%	43.3%	20.4%
cos sim loss	68.0%	84.7%	75.4%	78.5%	31.0%	44.4%	21.3%

Another try is to only calculate and minimize the cosine similarity loss. The cosine similarity losses between known samples to their known labels, as well as the cosine similarity between unknown samples to the selected candidate labels, are minimized:

$$\begin{aligned}
 &\text{if: } (\cos \theta_{\max,1} \geq \theta_{\text{confidence, known}}) && \text{then: } \mathcal{L} = 1 - \cos \theta_{\max,1} \\
 &\text{else if: } (\cos \theta_{\max,2} \geq \theta_{\text{confidence, known}}) && \text{then: } \mathcal{L} = 1 - \cos \theta_{\max,2} \\
 &\text{else:} && \mathcal{L} = 0
 \end{aligned}$$

This loss calculation does not “push” unknown samples away from known classes, but draw them closer to their matched candidates. In this way, it also encourages to select again those in previous batch already selected labels. In other words, it encourages cluster forming across batches.

The result shows, again as a linear adaptation method, it does not help much for unknown sample detection. H-score remains about the same. However, even when the unknown accuracy slightly reduces, it still achieves a better performance during cluster forming.

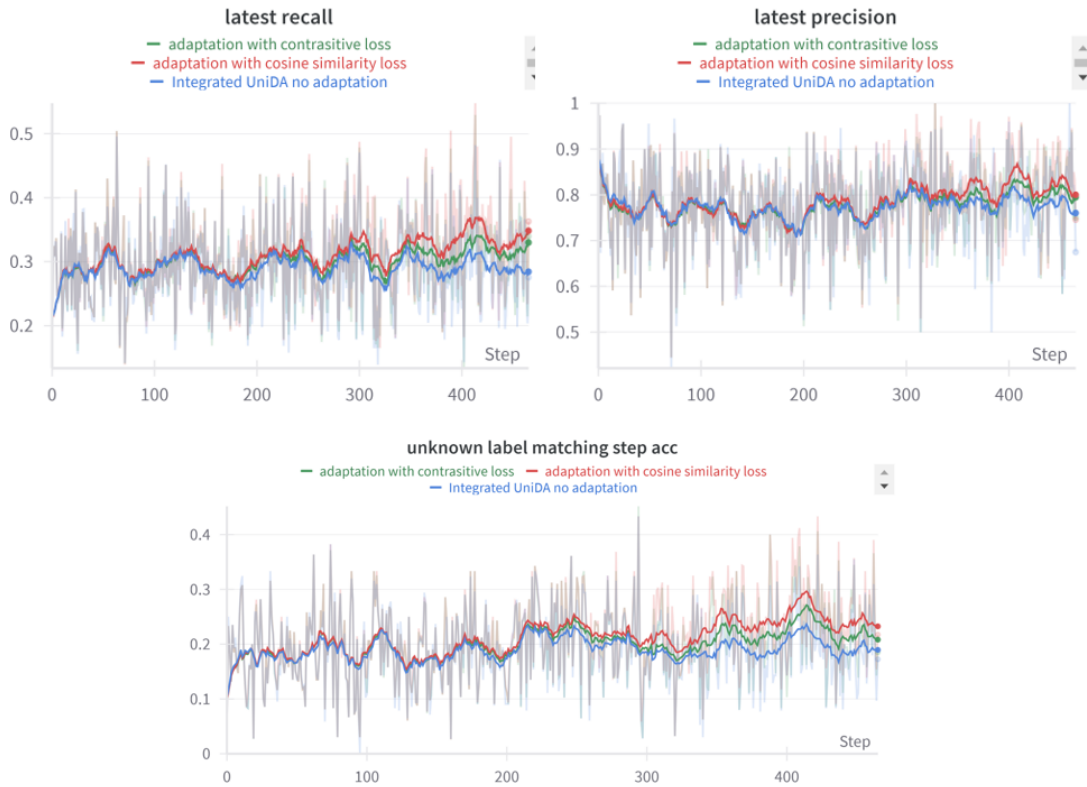


Figure 5.3.: Adaptation during Integrated UniDA

The pictures 5.3 above also prove the result that the adaptation helps with unknown cluster forming. Due to the initializing method of the linear adapter, the output of CLIP remains unchanged and the performance are the same with the solution integrated UniDA without any adaptation (marked with blue). While along with the adaptation process, both with contrastive loss (marked with green) and with cosine similarity loss (marked with red), the performance of unknown clusters forming is getting better. And the improvement by only cosine similarity loss minimization based adaptation is the greatest.

Considering of each batch in the data stream is only processed once, the improvement in this “single epoch” adaptation is quite considerable.

5.3. Not Distance Based Method

The discussion in this chapter up to now, is to perform classification and clustering based on label matching result. And label matching is only done based on distance in CLIP feature space.

However, label selection itself, is essentially a classification task, which can be solved with many other methods in addition to distance-based matching.

GPT2 in section 4.1 is exactly an example. It seems that GPT is generating labels without any candidates dictionary. But it is only because its dictionary is too big which includes over 50 thousands tokens. After all it is still a classification task, solved by transformer decoder. If we would like to have a label with 2 tokens as described in chapter 4, then it is a classification with 50k times 50k classes.

In section 4.2, a candidates set which includes up to 40 thousands label candidates are used for distance based label matching. And result shows that the ability of distance based solution is weaker if the size of candidates set grows over 10 thousands. If we combine these two solution with each other, it is exactly the concept in this section, which is to use a classifier and together with a candidates set instead of distance based label matching. Due to the large size of the candidates set, the classifier is usually a decoder-only transformer.

To build up such a dictionary and to train such a transformer decoder are both big challenges. This paper does not address any development in these aspects. However, many researches have reached this topic. A recently pulished method NOVIC developed an ‘object decoder’ which is able to select labels from a dictionary including 42919 nouns. And this decoder-only transformer also uses CLIP to encode the images. This transformer is much lighter than GPT2 and works better than distance based solution. [14]

Also the training approch is improved. In section 4.1., a MLP between CLIP and GPT2 is trained with image and true label text pairs from ImageNet. While NOVIC uses only generated text and key word pairs for training, because the CLIP is also an encoder for texts. This is much more efficient than a image based training.

6. Conclusion

Following is an overview of this thesis. In this chapter, a conclusion is drawn.

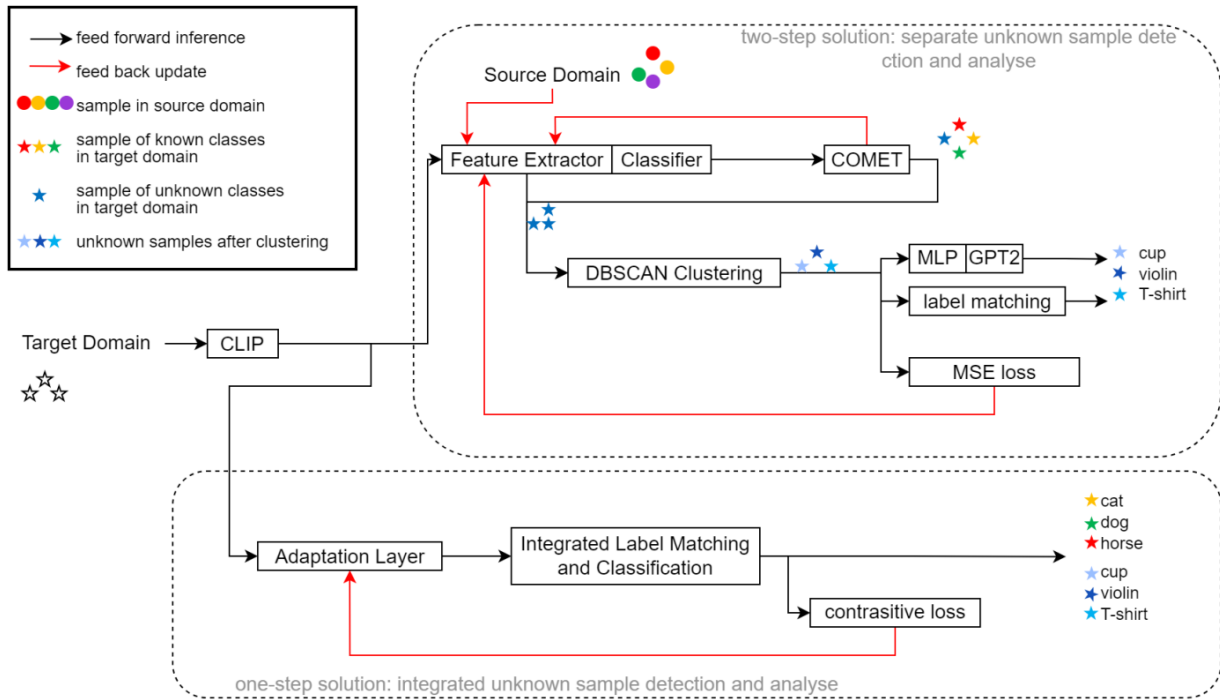


Figure 6.1.: Identifying Unknown Classes: An Overview

6.1. One-Step vs. Two-Step

In the frame of online source-free UniDA problems and unknown class identification, as described in the figure 1.1 in chapter 1, the workflow begins from classification and rejecting samples as unknown, followed by detected unknown samples clustering and label generation.

This thesis brought up 2 approaches. One is to separate unknown sample clustering and label generation as 2 steps (CH3 and CH4), while the other is to integrate clustering into label matching (CH5).

As the overview on figure 6.1 shows, 2-step solution uses the method COMET for classification and unknown sample detection. Then it implement DBSCAN for a online unknown samples clustering, after which a label is generated. As the red line shows, the adaptation also happens twice, once for contrastive learning between known and unknown samples, the other time for learning the features between different unknown clusters.

And 1-step solution combines unknown sample clustering with label matching, even together with classification and unknown sample detection.

The result and argument in previous chapters are based on real domain in DomainNet as source domain and painting as target. Here in the table 6.1, a more direct comparison and more results based on other datasets can be found.

Table 6.1.: one-step vs. two-step

method	known	UniDA acc		Unknown Clusters				
		unknown	H-score	precision	recall	f1-score	noise ratio	label acc
DomainNet painting to real								
one-step1	68.0%	84.7%	75.4%	78.5%	31.0%	44.4%	0%	21.3%
two-step1	55.7%	67.3%	61.0%	86.4%	94.3%	90.2%	76.6%	-
two-step2	55.7%	67.5%	61.0%	89.2%	61.8%	73.0%	71.6%	-
DomainNet real to painting								
one-step1	36.8%	90.7%	52.4%	26.7%	11.9%	16.5%	0%	11.6%
two-step1	38.4%	86.1%	53.1%	21.5%	67.9%	32.7%	84.3%	-
VisDA								
one-step2	77.7%	78.8%	78.2%	90.6%	4.8%	9.1%	0%	1.6%
one-step3	88.3%	55.5%	68.2%	96.7%	45.4%	61.8%	0%	38.2%
two-step3	57.0%	64.4%	60.5%	55.3%	86.0%	67.3%	54.2%	-
two-step4	56.9%	64.8%	60.6%	27.0%	82.2%	40.7%	31.1%	-

In this table, the method with different hyperparameter are marked with different index. For example in the experiments of DomainNet painting to real adaptation, two different hyperparameters set up for two-step solution, two-step1 and two-step2, show the balance between noise ratio and f1-score. Results from two-step3 and two-step4 show this trade off again on VisDA dataset.

The results based on DomainNet real to painting adaptation show, that the one-step solution is very sensitive for hyperparameter tuning. Usually when hyperparameter is tuned, like DomainNet painting to real adaptation or VisDA, one-step solution has better performance. But if the same hyperparameter is used for different domains, the influence for one-step solution is bigger.

It is argued that DBSCAN in two-step solution and label matching in one-step solution in this thesis are both distance based method, which is the reason why they are sensitive for hyperparameter tuning. While two-step solution uses a classifier for classification and unknown sample detection, which is better at this aspect. Even though with distance based solution, further improvement can also be done like implementing a dynamic adaptation.

The result also shows at first glance, that two-step solution has a better clustering result but high noise ratio. However, one-step can also have a non-equal-to-zero noise ratio by implementing a confidence threshold. While on the other hand, it is very difficult for DBSCAN clustering to further reduce the noise ratio. From this point of view the performance of one-step is still better.

The experiments result based on VisDA shows another issue of one-step solution. The setup for one-step1 and one-step2 are using a dictionary with 10k label candidates. It fits more to

DomainNet which has 345 classes. As for VisDA, it has only 12 classes. And those classes themselves are like “bigger categories”. For example “plant” can include “tree”, “grass”, “flower” and so on. A dictionary which is too big has no profit for this experiment at all. When the dictionary size reduces to only 1000, the result with setup one-step3 gets better.

6.2. Adaptation efficiency

In this thesis, many adaptation methods are brought up as the red lines shown on the figure 6.1.

As described in section 3.5 and 3.6, the second adaptation in two-step solution is updating parameters in one-layer feature extractor to learn the feature between different unknown clusters. The result shows that the adaptation efficiency is quite low.

And section 5.2 introduced another adaptation which is to update parameters in a linear adaptation layer. The result shows a pretty good adaptation efficiency.

The adapter in section 5.2 doesn’t need pre-training. With a special initialization method, the input and output of this adapter are exactly the same. While the feature extractor in chapter 3 do need pre-training. When it is asked to learn some new knowledge during adaptation, such a feature extractor has a relative lower adaptation efficiency.

And in many cases, adaptation efficiency and zero-shot learning ability are contradictory. Because zero-shot learning requires usually a large foundational model which is trained on large scale of data. And such a model usually have quite low adaptation efficiency. Even fine tuning will require a large dataset and many epochs learning.

The linear adaptation layer in section 5.2 with its one-hot coding initialization also solved this contradictory. It performs adaptation directly in output space and is very light comparing with the original foundational model. In fact, it can also be used in the two-step solution to replace the feature extractor.

In this thesis, this adaptation layer is linear, which can only achieve linear feature mapping. However, similar non-linear single layer adapters can also be built and developed to ensure identity initialization and gradual deviation through training.

For example:

$$\mathbf{x}_2 = \text{ReLU}(W_1 \mathbf{x}_1 + b_1) - \text{ReLU}(W_2 \mathbf{x}_1 + b_2) + \mathbf{x}_1$$

with initialization: $W_1 = W_2, b_1 = b_2$

When such a layer is built and initialized, and during the adaptation W_1, b_1 are frozen, only W_2, b_2 are updated, then it is an example of a non-linear single layer adapter.

Further research can be done to build such adapters and to evaluate their efficiency.

6.3. Label Generation vs. Selection

In this thesis around topic of label generation and selection, many methods have been mentioned. Section 4.1 uses GPT2 to generate label text. Section 4.2 select a label from

a candidates set based on distance. And section 5.3 introduced a newly published research to achieve so-called “Open Vocabulary Image Classifier”.

Generation or selection, candidates dictionary or open vocabulary, even they seem to be different approaches at first glance, they are after all the same to finish classification tasks. The only difference and also the biggest challenge at the same time is scale. Methods like distance based label matching are not able to achieve good performance in a large scale application.

The following table shows a comparison between these approaches.

Table 6.2.: After all, they are solutions for the same classification task [10] [14]

	GPT2 generation	label selection	NOVIC
Target Number	50257 ² (for 2-token label)	5000 - 20000	42919
Architecture	LLM (Decoder-only Transformer)	distance based	Decoder-only Transformer
Training Effort	Mapping Network 8M	no training needed	12.2M

Nowadays, the scale of models are getting larger and larger. GPUs around the world are calculating huge amount of data. AGI seems around the corner. Images, texts as well as proteins, mechanical structures, everything can be encoded. All that exists seem to be nothing but data. And certainty is nothing but statistics. We are now training AIs, while algorithm from social media is also influencing our thinking. “Man is a thinking reed.” At the very end of this thesis, I shall remind myself again to believe in reason and serve humanity.

A. Additionally: German Summary

Viele Methoden der Universal Domain Adaptation (UniDA) wurden entwickelt, um ein Modell vom source domain auf den target domain anzupassen. Sie können sowohl das Problem des domain shift als auch des category shift lösen. Allerdings behandeln sie alle abgelehnten Sample nur als eine einzige Unknown Class während der Klassifikation.

Heutzutage ist es jedoch mit Foundation Models möglich, Open-Class Classification oder Image Label Generation mithilfe von Zero-shot Learning durchzuführen. Diese Arbeit verbindet beide Themen und nutzt Zero-shot Learning, um einen Schritt weiter als UniDA zu gehen – nämlich zur Unknown Class Identification, einschließlich Unknown Sample Clustering und Cluster Label Text Generation.

List of Figures

1.1. Identifying Unknown Classes: A Workflow	2
2.1. 6 domain of DomainNet, reproduced from [6]	4
3.1. CLIP used as backbone to draw features from images	10
3.2. An exsample of DBSCAN in 2-dimensional feature space	11
3.3. Online Clustering with DBSCAN	12
3.4. Using unknown sample clustering result for adaptation	13
3.5. noise ratio remains high during unknown sample clustering	16
4.1. Generate labels for unknown clusters after DBSCAN	17
4.2. Training procedure of a MLP between CLIP and GPT2	20
4.3. Try to speed up training procedure by cosine similarity loss	20
5.1. label matching in UniDA problems	25
5.2. More adaptation possibility by contrasitive learning	28
5.3. Adaptation during Integrated UniDA	30
6.1. Identifying Unknown Classes: An Overview	33

List of Tables

2.1.	select classes from DomainNet to set up catagory shift [5]	3
2.2.	select classes from VisDA to set up catagory shift [5]	4
3.1.	hyperparameters tuning for DBSCAN based unknown sample clustering . . .	14
3.2.	Performance of unknown sample clustering after UniDA	15
3.3.	unknown sample clustering result for learning	15
4.1.	Labels cover rate of dataset by candidates	22
4.2.	Top-3 accuracy of label matching	22
5.1.	Performance of integrated UniDA	27
5.2.	Performance of integrated UniDA with adaptation	29
6.1.	one-step vs. two-step	34
6.2.	After all, they are solutions for the same classification task [10] [14]	36

Bibliography

- [1] K. You, M. Long, Z. Cao, J. Wang and M. I. Jordan, “Universal domain adaptation,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2715–2724.
- [2] H. Larochelle, D. Erhan and Y. Bengio, “Zero-data learning of new tasks,” in *AAAI Conference on Artificial Intelligence*, 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:7249642>
- [3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, “Learning transferable visual models from natural language supervision.” [Online]. Available: <http://arxiv.org/pdf/2103.00020>
- [4] J. N. Kundu, N. Venkat, R. M. V and R. V. Babu, “Universal source-free domain adaptation.” [Online]. Available: <http://arxiv.org/pdf/2004.04393>
- [5] P. Schlachter and B. Yang, “Comet: Contrastive mean teacher for online source-free universal domain adaptation.” [Online]. Available: <http://arxiv.org/pdf/2401.17728>
- [6] X. Peng, Q. Bai, X. Xia, Z. Huang, K. Saenko and B. Wang, “Moment matching for multi-source domain adaptation.” [Online]. Available: <http://arxiv.org/pdf/1812.01754>
- [7] X. Peng, B. Usman, N. Kaushik, J. Hoffman, D. Wang and K. Saenko, “Visda: The visual domain adaptation challenge.” [Online]. Available: <http://arxiv.org/pdf/1710.06924>
- [8] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Knowledge Discovery and Data Mining*, 1996. [Online]. Available: <https://api.semanticscholar.org/CorpusID:355163>
- [9] R. Mokady, A. Hertz and A. H. Bermano, “Clipcap: Clip prefix for image captioning.” [Online]. Available: <http://arxiv.org/pdf/2111.09734>
- [10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, “Language models are unsupervised multitask learners,” 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:160025533>
- [11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [12] X. Liu, Y. Zhou, T. Zhou, C.-M. Feng and L. Shao, “Coca: Classifier-oriented calibration via textual prototype for source-free universal domain adaptation.” [Online]. Available: <http://arxiv.org/pdf/2308.10450>
- [13] G. A. Miller, “Wordnet: a lexical database for english,” vol. 38, no. 11, 1995. [Online]. Available: <https://doi.org/10.1145/219717.219748>

- [14] P. Allgeuer, K. Ahrens and S. Wermter, “Unconstrained open vocabulary image classification: Zero-shot transfer from text to image via clip inversion,” *Winter Conference on Applications of Computer Vision (WACV)*. [Online]. Available: <http://arxiv.org/pdf/2407.11211>

Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, 31.Mar.2025

Zhengyu Bao